JASEEM QURAISHI

Full Stack Engineer | Indore, India (Open to relocate)
+91 9555917856 | jaseem1quraishi@gmail.com | linkedin.com/in/jaseem-quraishi/ | github.com/Zasim1074

PROFESSIONAL SUMMARY

Full Stack Engineer with experience developing and optimizing full stack production web applications using React, Next.js, and TypeScript, with hands-on experience in Python FastAPI and PostgreSQL. Skilled in REST API integration, authentication, performance optimisation, reusable component development, and deployment automation. Focused on building scalable, maintainable applications and delivering reliable end-to-end features.

WORK EXPERIENCE

Frontend Developer - FloorWalk Consultants Pvt. Ltd., Indore, India Aug 2025 - Present
Products & Services: Floorwalk, FloorInsights, floor.estate CRM, client/ auditor/ manager/ Moderator portals, Shopper app

- Engineered and shipped to production backend APIs for an AI report-analysis feature, lowering server costs by 25% through batching logic that avoids one LLM call per report submission.
- Constructed and presented the frontend of a real-estate CRM (Next.js, React, TypeScript) to 10–15 internal users, collaborating with stakeholders to implement role-based access control for modules including leads, properties, and dashboards.
- Architected a reusable API layer (caching, retries, centralized error handling) and custom hooks that replaced per-component fetch logic across modules, decreasing duplicate API calls by 30%.
- Implemented WebSocket-driven dashboards that push report updates, analytics and notification counts without page refresh, curtailing dashboard render times by 100ms using useMemo and useCallback.
- Reduced the production build size by approximately 81% (70 MB to 13 MB) by removing unused libraries and dead code, compressing images and adding lazy loading; practical load time improved from 6–8s to 3–4s.
- Redesigned the client portal and Shopper app; developed features across the auditor and manager portals and the company websites (floorwalk.in, nps.floorwalk.in); shipped multilingual apps (8+ languages).
- Converted Figma designs into reusable, component-based UI shared across 5+ modules and portals, diagnosing and resolving rendering bottlenecks with Chrome DevTools profiling.
- Collaborated with 3 backend engineers on API integration and debugging across 4 core authentication, file upload, and reporting flows.
- Integrated Auto deployment for 8 frontend portals and 2 React native mobile applications for iOS and Android both that shifted the whole deployment from manual to automate and reduced time around 3 hours to 30 minutes.

SKILLS

Languages: JavaScript (ES6+), TypeScript, Python, SQL, HTML5, CSS3, SQL

Frontend: React.js, Next.js (App Router), Redux, TanStack Query, Tailwind CSS, shadcn/ui, Axios, Responsive Design

Backend: FastAPI, SQLAlchemy 2.0, Pydantic v2, PostgreSQL, Alembic, REST API Design, JWT Authentication

Testing & Tools: Pytest, React Testing Library, Git, GitHub, Postman, Chrome DevTools

Deployment: Vercel, cPanel / WHM, AWS, Apple Developer Account, Google Play Console

EDUCATION

Guru Nanak Dev University Aug 2022 - Jul 2025
B.Sc., Information Technology Amritsar, India

PERSONAL PROJECTS

LLM Reliability Lab GitHub

- Built an LLM evaluation platform with FastAPI and Next.js featuring semantic caching, benchmark execution, experiment history and dashboards.
- Implemented deterministic LLM-as-a-judge evaluation workflows and bias diagnostics to analyze model output quality and consistency. Developed a RAG prompt-injection testing harness, provider timeout handling, authentication and automated backend tests.

TrackHire Link

- Designed a REST API with FastAPI and Pydantic v2 covering job applications, companies, interviews, resumes and activity tracking, backed by a normalized PostgreSQL schema (SQLAlchemy 2.0, Alembic migrations).
- Implemented JWT authentication with protected routes using FastAPI dependency injection, alleviating API latency by 25%; tested the API with Pytest and the UI with React Testing Library.
- Engineered the frontend with search, filtering, pagination and role-based UI using shadcn/ui, curtailing redundant API calls by 30%.